Práctica 2. Privacidad en los Sistemas Inteligentes (1 punto)

Importante:

1. Realizad la práctica en los mismos grupos de 3 personas que en la práctica anterior (P01).
2. Todos los integrantes del equipo deben estar familiarizados con la totalidad de los ejercicios de la práctica.
3. **IMPORTANTE:** No incluyáis en la memoria descripciones o conceptos que no comprendáis. Consultad cualquier duda con el profesor de prácticas.

Anonimización de datos (10 %)

1. **Ataque de enlace (*linkage attack*) sobre datos tabulares:** Demostrad la vulnerabilidad de las técnicas de enmascaramiento tradicionales frente a información auxiliar.
    - Cargad el dataset *Adult Census Income*[1] utilizando `pandas`.
    - Argumentad qué atributos del dataset podrían considerarse identificadores directos, cuasi-identificadores o atributos sensibles. Para identificadores y cuasi-identificadores, indicad más de una estrategia de anonimización que podría aplicarse.
    - Implementad una función de anonimización que garantice k-anonymity ($k = 3$) sobre los cuasi-identificadores: `age`, `sex` y `native_country`. Indicad la técnica de anonimización aplicada a cada atributo y justificad vuestra elección.
    - ¿Es suficiente con anonimizar solo uno de estos atributos? Comprobad el resultado de anonimizar únicamente `age` o únicamente `native_country` y explicad por qué no se garantiza k-anonymity en esos casos.
    - Se os proporciona un dataset auxiliar externo ficticio `external_dataset.csv`. Explicad, a nivel teórico, cómo un atacante podría utilizar este dataset para realizar un ataque de enlace (*linkage attack*) y reidentificar a individuos del dataset anonimizado.
    - Programad un script que realice el *linkage attack*: cruzad el dataset externo con el dataset anonimizado utilizando los cuasi-identificadores para reidentificar inequívocamente, cuando el cruce devuelva un único registro, la categoría del salario (`income`) junto con el nombre y el número de pasaporte del individuo.
    - Documentad en la memoria las capturas de pantalla de los registros reidentificados y explicad por qué k-anonymity falla en este escenario. ¿Podría haberse evitado este ataque aplicando l-diversity o t-closeness? Justificad vuestra respuesta.

[1] `https://archive.ics.uci.edu/ml/datasets/adult`

Federated Learning (50 %)

2. **Simulación base de Federated Learning (FL):** Implementad una simulación de un entorno de entrenamiento distribuido.
    - Particionad el subconjunto de entrenamiento del dataset CIFAR-10[2] en 5 particiones disjuntas (IID) para simular 5 clientes.
    - Instanciad 5 modelos locales y 1 modelo global utilizando una arquitectura CNN sencilla (por ejemplo, un modelo secuencial de 3 capas convolucionales) en PyTorch.
    - Programad el bucle de entrenamiento federado utilizando el algoritmo FedAvg. Realizad el entrenamiento durante 10 rondas de comunicación.
    - Evaluad y registrad el *accuracy* del modelo global sobre el conjunto de test de CIFAR-10 en cada ronda.
3. **Envenenamiento del modelo (*model poisoning*):** Vulnerad el servidor central.
    - Modificad el Cliente 5 para convertirlo en un nodo malicioso. Este cliente realizará un ataque de inversión de etiquetas (transformando la clase y en $(y+1) \pmod{10}$) y aplicará un *gradient scaling*: multiplicará los pesos de su modelo local por un factor de 100 antes de enviarlos al servidor.
    - Ejecutad el entrenamiento y generad una gráfica que muestre cómo el *accuracy* global se desploma al agregar el modelo envenenado con FedAvg.
4. **Mitigación aplicando agregación robusta bizantina:** Mitigad el ataque de envenenamiento del modelo aplicando una agregación robusta.
    - Modificad la función de agregación del servidor: sustituid FedAvg por *Trimmed Mean*, descartando el 20 % de los valores más altos y más bajos para cada parámetro.
    - Repetid el ataque y demostrad, mediante una gráfica comparativa, cómo el servidor ignora ahora la actualización maliciosa y el modelo converge.

Differential Privacy (40 %)

5. **Ataque de inferencia de membresía (*Membership Inference Attack*, MIA):** Evaluad la fuga de privacidad del modelo global del ejercicio anterior.
    - Buscad información sobre el ataque de inferencia de membresía (*Membership Inference Attack*, MIA) y explicad en qué consiste y cómo funciona.
    - Utilizando el framework Adversarial Robustness Toolbox (ART)[3], instanciad un ataque `MembershipInferenceBlackBox`.
    - Entrenad el modelo de ataque utilizando predicciones del modelo global sobre un subconjunto de datos de entrenamiento del Cliente 1 y datos del conjunto de test.
    - Evaluad el ataque y calculad la *precision* y el *recall* del atacante al intentar adivinar qué imágenes pertenecían al dataset privado del Cliente 1. Documentad estos valores.

[2] `https://www.cs.toronto.edu/~kriz/cifar.html`
[3] `https://adversarial-robustness-toolbox.readthedocs.io/`

6. **Mitigación con privacidad diferencial (*Differential Privacy*, DP):** Proteged los datos de los clientes honestos frente a la memorización utilizando *Differential Privacy Stochastic Gradient Descent* (DP-SGD).
    - Integrad la librería Opacus[4] en el bucle de entrenamiento local del Cliente 1.
    - Configurad el motor de privacidad (`PrivacyEngine`) con un recorte máximo del gradiente (*max_grad_norm*) de $C = 1{,}0$.
    - Estudio de ablación: realizad el entrenamiento local variando el nivel de ruido para forzar dos presupuestos de privacidad distintos: $\epsilon = 3{,}0$ (alta privacidad) y $\epsilon = 8{,}0$ (privacidad media), manteniendo $\delta = 10^{-5}$. Repetid el ataque MIA del ejercicio anterior sobre el modelo global tras aplicar DP-SGD en el Cliente 1.
    - Elaborad una tabla final que contraste el *accuracy* limpio del modelo y el éxito del ataque MIA (*precision*/*recall*) en los 3 escenarios: sin DP, DP con $\epsilon = 8{,}0$ y DP con $\epsilon = 3{,}0$.

Entrega:

- Fecha límite de entrega: **lunes, 27 de abril de 2026, a las 10:30 h**.
- Entrega a través del Campus Virtual (Moodle). Solo una entrega por grupo.
- Solo se admiten los siguientes archivos:
    - **Memoria** (formato .pdf).
        - Extensión máxima: 8000 palabras (la portada, el índice y el apartado de referencias no cuentan).
        - Nombre: `<GN>_memoria.pdf`. Donde `<GN>` es el número de grupo.
          Por ejemplo: `G2.03_memoria.pdf` para el grupo G2.03.
    - **Código fuente** (Jupyter Notebooks/Scripts Python; formato .ipynb/.py).
        - Un fichero Jupyter Notebook por cada ejercicio que requiera ejecución de código. Todos los resultados obtenidos deben incluirse en el notebook y ser perfectamente reproducibles sin errores.
        - Adicionalmente, y de forma opcional, se admiten ficheros Python (.py) para la organización del código (funciones, clases, módulos, etc.), así como para facilitar el desarrollo colaborativo en repositorios Git.
        - Nombre: `<GN>_codigo_<EjN>.ipynb`. Donde `<GN>` es el número de grupo y `<EjN>` es el número de ejercicio.
          Por ejemplo: `G2.03_codigo_2.ipynb` para el segundo ejercicio del grupo G2.03.

Defensa

Posteriormente a la entrega, el profesor indicará una fecha para la defensa de la práctica. En la defensa deberán estar presentes todos los integrantes del grupo, cada uno con su ordenador y con todo lo necesario para poder mostrar cualquier ejercicio de la práctica.

Durante la defensa, cualquiera de los miembros del grupo podrá ser requerido para explicar cualquier apartado, demostrar su ejecución o responder preguntas relacionadas.

[4] `https://opacus.ai/`

Evaluación

La evaluación de cada apartado tendrá en cuenta tanto el contenido de la memoria como las respuestas y la destreza mostradas en la defensa. Tened en cuenta que, si durante la defensa el grupo no es capaz de justificar el contenido que figura en la memoria, podría invalidarse el apartado correspondiente.

Adicionalmente, se valorará positivamente la participación en clase, que podrá suponer hasta un 15 % de puntuación adicional.